

Especificação Léxica Checkpoint 1

Construção de Compiladores

prof. Marcelo Maia

Ana Alice Cordeiro de Souza - 12211BCC028

Anna Karolyna Pereira Santos - 12221BCC046

1. Um exemplo da nossa linguagem

Antes de definir as regras, decidimos como um programa da nossa linguagem se parece. Escolhemos uma sintaxe parecida com a do slide 5 da Aula 1, com **fun** para funções e **var** para variáveis, e anotação de tipo depois de **:**.

```
fun dobro(x: int): int {  
    return x * 2;  
}  
fun main(): void {  
    var n: int = 21;  
    var msg: string = "resultado ok";  
    var achou: bool = false;  
  
    if (n >= 10) {  
        while (n > 0) {  
            n = n - 1;  
        }  
    } else {  
        achou = true;  
    }  
    // comentário de linha  
    /* comentário de bloco */  
    print(msg);  
}
```

Esse é basicamente o programa que usamos pra testar o scanner, é o mesmo do Main.java.

2. Alfabeto e case-sensitivity

A linguagem usa só ASCII: letras (a-z, A-Z), dígitos (0-9), underscore (_) e os símbolos de operador/pontuação listados abaixo. Não aceita acento nem caractere especial em identificador se aparecer, o scanner trata como erro léxico.

É case-sensitive: total e Total são identificadores diferentes. Se alguém escrever IF em vez de if, isso não vira a palavra reservada, o scanner reconhece como identificador comum (quem vai reclamar disso depois é a análise semântica, não o léxico).

3. As cinco categorias de token

Identificador:

letra (letra | dígito | '_')*, ou seja, tem que começar com letra, depois pode ter letra, dígito ou “_” à vontade. Não colocamos limite de tamanho. Exemplos: total, contaItens, taxa_juros, x1.

Palavra reservada:

Usamos a mesma regra do identificador só que, depois de reconhecer o texto, comparamos com uma lista fixa de palavras-chave, é a abordagem “identificador + tabela” que foi mostrada em aula, em vez de escrever uma regra separada pra cada palavra. Nossa lista tem 14 palavras:

```
fun    var    if     else   while  return
int    double bool  char   string void
true   false
```

String:

Começa e termina com ". No meio pode ter qualquer caractere, menos outra aspa dupla, que fecha a string, e menos quebra de linha (nossa linguagem não deixa string ocupar mais de uma linha, achamos mais simples assim pro checkpoint. Também aceitamos escape (\", \n, \t, \\) decidimos incluir isso porque senão nem dava pra escrever uma string que precisa ter aspas dentro dela.

Operador:

Trabalhamos com operadores de 1 ou 2 caracteres, decidindo pelo “casamento mais longo”, se o próximo caractere formar um operador válido de 2, o scanner junta os dois, senão fica só com 1:

- Atribuição/igualdade: ==
- Relacionais: < <= > >= !=
- Aritméticos: +- * / %
- Lógicos: ! && ||

- Delimitadores (pontuação que a linguagem também precisa pra funções, blocos e listas): () { } , ; :
- O : é usado pra anotação de tipo (x: int), e não tinha como deixar de fora porque toda a nossa sintaxe de função depende dele.

Literal numérico:

dígito+ ("." dígito+)? → um inteiro é só dígitos, um double tem um ponto no meio com dígitos dos dois lados. Não implementamos notação científica nem hexadecimal (não era obrigatório e achamos que ia complicar o scanner sem necessidade pra esse checkpoint). Uma decisão que discutimos bastante: o que acontece se alguém digitar 3. sem nenhum dígito depois do ponto? Decidimos que isso é erro léxico, não que o scanner “desiste do ponto” e trata como 3 seguido de outro token. Uma vez que ele viu o ., ele já assume que vai formar um double.

4. Comentários e espaço em branco

Espaço, tab, \r e \n são só descartados, não geram token. Comentário de linha é // até o fim da linha, ou até o arquivo acabar, se o comentário for a última coisa do arquivo, nesse caso não tem erro nenhum, simplesmente não tem mais nada pra ler depois. Comentário de bloco é /* ... */ , pode passar de uma linha pra outra. Decidimos que não é aninhado, o primeiro */ que aparecer fecha o comentário, mesmo se tiver um /* escrito dentro por engano. Se o arquivo terminar sem fechar o comentário, isso é erro léxico.

5. O que acontece quando dá erro

A regra principal que seguimos: o scanner nunca pode travar o programa com uma exceção. Se algo der errado, ele registra o erro, com mensagem, linha e coluna, numa lista interna e segue tokenizando o resto do arquivo normalmente, dá pra ver isso funcionando nos testes que a gente rodou (ScannerTest, 11/11). Situações de erro que cobrimos:

- Caractere que não existe na linguagem (tipo @) — descarta e segue.
- String que não fecha até o fim da linha ou até o fim do arquivo.
- Comentário de bloco que não fecha até o fim do arquivo.
- . sem dígito depois num número.

➤ & ou | sozinhos (só existem como && e ||).

Em todos os casos, reportamos a linha/coluna de onde o problema começou, por exemplo, onde a string abriu, não de onde o scanner percebeu o erro, achamos que assim fica mais fácil pra quem for corrigir o próprio código depois.